check_cm_runtime_deps_parity

scripts/pre_build/check_cm_runtime_deps_parity.sh — CM-RUNTIME-DEPS-PARITY

Revision: 1 **Last modified:** 2026-08-21T19:20:00Z **Status:** active
Item: BOB-154 (host venv and production container ran different stacks)

Overview

Asserts that the interpreter and dependency set the **tests** run against is the same one **production** serves. It compares the two REAL resolved sets — read live out of the host virtualenv and live out of the running container — never a requirements file against itself. A specification compared to itself is a check that cannot fail.

Not yet wired into `scripts/pre_build_verification.sh`; the wiring block is at the end of this document and is applied by the operator (that file is contended).

Why it exists — the forensic anchor (BOB-154, measured 2026-08-21)

`download-proxy/requirements.txt` carried `>=` floors with no ceiling and no lock, and `start-proxy.sh` pip-installs from it into a stock `python:3.12-alpine` image **at every container start**. There is no image build and no baked dependency layer, so the resolved set was a function of the install *date* rather than of the file. Measured from `dist-info` mtimes:

```
host .venv  installed 2026-08-07 23:12  ->  CPython 3.14.6,
starlette 1.4.1
container   installed 2026-08-21 15:20  ->  CPython 3.12.13,
starlette 1.6.0
```

Eight packages diverged — exactly those that shipped a release in the fourteen days between the two installs — plus a **two-minor interpreter gap** that the original report did not see at all. Every green suite was evidence about a stack nobody served.

The harm was not hypothetical. Running the suite on the production interpreter for the first time surfaced `tests/conftest.py` binding to `asyncio.events._get_event_loop_policy`, a private API that does not exist before Python 3.13. The test suite had silently accreted a dependency on an interpreter production does not run, because nothing ever checked.

Prerequisites

- `bash` 4+ (associative-array-free, but uses `mktemp`, `awk`, `grep`).
- For a non-SKIP run: `podman` or `docker`, the `qbittorrent-proxy` container running, and a host virtualenv at `.venv/` whose interpreter has `pip`.
- No network. No writes. The container is only ever read via `exec`.

Usage

```
bash scripts/pre_build/check_cm_runtime_deps_parity.sh
bash scripts/pre_build/check_cm_runtime_deps_parity.sh --help
```

Exit codes: `0` PASS or honest SKIP, `1` FAIL, `2` usage ERROR. The verdict line is always last on stdout so the pre-build wiring can read it with `tail -n1`; a SKIP verdict begins with `SKIP(`.

Consumer DATA via environment (§11.4.35):

Variable	Default	Purpose
`VENV_PYTHON`	`.venv/bin/python`	host interpreter to probe
`PROXY_CONTAINER`	`qbittorrent-proxy`	container to probe
`CONTAINER_RUNTIME`	autodetect, podman first	runtime binary

`CM_DEPS_HOST_SNAPSHOT` / `CM_DEPS_CONTAINER_SNAPSHOT` inject pre-recorded snapshots instead of probing. They exist **only** so the paired meta-test can build hermetic fixtures; setting them in a real run defeats the entire point of the gate, which is that it reads reality.

Internal behaviour

1. Probe both sides into a normalised snapshot: a `#python X.Y.Z` line plus `name==version` lines, names normalised per PEP 503 (`typing_extensions`, `typing-extensions` and `Typing.Extensions` are one package).
2. Refuse an empty snapshot. A blind probe and a matching stack both return a quiet zero, and reading that as parity is the §11.4.201(6) false-null.
3. Run the **control needle** (§11.4.201(7)(b)): re-run the comparator against a copy of the container snapshot carrying one synthetic package the host cannot have, and require the finding count to rise by exactly one. Silence from a comparator never proven able to speak is not evidence.
4. Compare, and classify each finding as a failure or a declared divergence.

What is compared, and what is deliberately not

Compared	Rule
Interpreter `major.minor`	must match; a minor gap changes language and stdlib behaviour
Interpreter patch	reported, **not** fatal — see below
Packages in both sets	versions must match exactly
Packages only in the container	FAIL — the tests cannot exercise a production dependency they do not have
Packages only in the venv	ignored — a venv is *supposed* to carry pytest, ruff, mypy, playwright
`pip`, `setuptools`, `wheel`	excluded by name

The patch-level and installer exclusions are deliberate applications of §11.4.201(1), which forbids a false-positive refusal as firmly as a false pass. Alpine’s CPython point release and the host’s will legitimately differ by days, and `download-proxy/requirements.txt` deliberately does not pin `pip` (pinning the installer inside the file it is installing is a self-modification

footgun). Failing on either would demand an action the fix cannot perform — and a gate that cries wolf is switched off within a week, which is a worse outcome than the skew it was reporting.

The installer exclusion is a **named list**, never a pattern, so it cannot silently swallow a real dependency. Fixture `golden-bad-installer-only` proves `pip` is ignored in the same run where a genuine skew is still caught.

Declared divergences, and why they cannot become an off switch

The BOB-154 acceptance criterion permits a divergence that is *declared* rather than eliminated. `DECLARED_DIVERGENCES` in the gate holds `key|reason|tracked-item` entries; a matching finding is printed loudly on every run and does not fail the gate.

Two properties keep that honest:

- every entry carries a reason and a tracked item, so it is debt with an owner rather than a shrug;
- a **stale** declaration — one matching no current divergence — is itself a FAIL. A declaration cannot be filed pre-emptively and cannot outlive the condition it excuses. This is the §11.4.227 monotone ratchet applied to excuses.

Honest SKIP

No container runtime, no running container, or no host interpreter produces a `SKIP(§11.4.3)` with the reason stated, and exit 0. A stopped stack is a legitimate state, not evidence of drift. A gate that hard-failed on a stopped stack would be disabled by the first person who ran a build with the stack down — so the SKIP is what keeps the gate alive. It is never silent: the reason is printed and the verdict line says `SKIP`, so the wiring can tell a skip from a pass.

Relationship to the pin

`download-proxy/requirements.txt` is now pinned to the full transitive closure, derived from the live container. The pin and this gate do different jobs and neither substitutes for the other:

- the **pin** makes production reproducible across restarts;
- the **gate** makes any future divergence visible.

A pin alone cannot notice that the *other* side moved, cannot reach the interpreter, and rots silently when nobody maintains it. A gate alone leaves production re-rolling its dependency dice on every restart.

A rotted pin is noticed by machinery that already exists: `scripts/scan.sh` runs `pip-audit -r download-proxy/requirements.txt`. Against the old floors pip-audit had to resolve to "latest" — a moving target nobody was running. Against exact pins it audits the production set and names the version to raise.

Edge cases

- **Stack down** — SKIP with reason, exit 0.
- **Container running but `pip` missing inside it** — probe fails, SKIP with reason rather than a fabricated verdict.
- **Empty snapshot from either side** — FAIL, never pass.
- **Blinded comparator** — FAIL via the control needle. The paired meta-test mutates the comparator to report nothing and proves the gate still fails.
- **Both sides pinned to the same rotted set** — the gate PASSes, correctly: they *do* agree. Rot is `pip-audit`'s job, not this gate's.

Verification

```
bash tests/pre_build/test_check_cm_runtime_deps_parity.sh
```

11 hermetic fixtures, one skip case, one §1.1 paired mutation, and a real-tree smoke run. The real-tree case deliberately does not assert a fixed exit code: it is 1 while the venv is unreconciled and 0 afterwards, and asserting either would make the harness lie on the other side of the operator's apply step.

Related

- `download-proxy/requirements.txt` — the pinned set this gate keeps honest
- `tests/pre_build/test_check_cm_runtime_deps_parity.sh` — the §1.1 pair
- `scripts/pre_build/check_cm_healthcheck_covers_served_ports.sh` — sibling gate, same §11.4.201 guard-honesty discipline
- `scripts/scan.sh` — `pip-audit` over the pinned file